

TrackServe Project

Work Breakdown Structure (WBS)

16-Week Development Timeline

1. Project Initiation & Planning (Weeks 1-2)

1.1 Project Kickoff

- 1.1.1 Team onboarding and role assignment
- 1.1.2 Project scope and objectives review
- 1.1.3 Timeline and milestones definition

1.2 Infrastructure Setup

- 1.2.1 Repository setup (Git, version control)
- 1.2.2 Development environment configuration
- 1.2.3 CI/CD pipeline setup
- 1.2.4 Staging and production servers

1.3 Documentation

- 1.3.1 Technical architecture document
- 1.3.2 API specification template
- 1.3.3 Code style guide and standards

2. Requirements & Design (Weeks 2-4)

2.1 Product Definition

- 2.1.1 User personas and stories
- 2.1.2 Feature prioritization (MVP vs. Phase 2)
- 2.1.3 Success metrics and KPIs

2.2 UX/UI Design

- 2.2.1 Information architecture and sitemap
- 2.2.2 Visual design system and brand guidelines
- 2.2.3 Component library design
- 2.2.4 Accessibility audit and guidelines

2.3 Technical Specifications

- 2.3.1 System architecture design
- 2.3.2 Database schema design
- 2.3.3 API endpoint specifications
- 2.3.4 Security and authentication approach
- 2.3.5 Third-party integrations mapping

2.4 Data Strategy

- 2.4.1 Data classification and governance
- 2.4.2 Analytics and tracking plan

3. Frontend Development (Weeks 5-12)

3.1 Core Components Development

- 3.1.1 Layout components (Header, Sidebar, Footer)
- 3.1.2 Reusable UI components (Button, Card, Badge, etc.)
- 3.1.3 Form components and validation
- 3.1.4 Modal and notification components

3.2 Feature Development

- 3.2.1 Voting and polling widgets
- 3.2.2 Civic issues browsing
- 3.2.3 Community map interface
- 3.2.4 Search and filtering functionality
- 3.2.5 Discussion and comments threads

3.3 State Management

- 3.3.1 Redux/Context setup and configuration
- 3.3.2 Store structure and actions
- 3.3.3 Data fetching and caching logic

3.4 Responsive Design

- 3.4.1 Mobile layout implementation
- 3.4.2 Tablet and desktop optimization
- 3.4.3 Cross-browser testing

3.5 Performance Optimization

- 3.5.1 Code splitting and lazy loading

3.5.2 Image optimization

3.5.3 Bundle size reduction

3.6 Analytics Integration

3.6.1 User tracking setup

3.6.2 Event logging implementation

4. Backend Development (Weeks 5-12)

4.1 API Development

4.1.1 Event endpoints (CRUD operations)

4.1.2 Civic issue endpoints

4.1.3 Voting/polling endpoints

4.1.4 Comment/discussion endpoints

4.1.5 Filter endpoints

4.1.6 Map data endpoints

4.1.7 Content endpoints

4.2 Authentication & Authorization

4.2.1 Anonymous session management

4.2.2 Rate limiting and abuse prevention

4.2.3 Security headers implementation

4.2.4 Input validation and sanitization

4.3 Business Logic

4.3.1 Content management logic

4.3.2 Voting and aggregation logic

4.3.3 Content moderation logic

4.3.4 Notification/alert logic

4.4 Error Handling & Logging

4.4.1 Centralized error handling

4.4.2 Logging system setup

4.4.3 Error monitoring and alerts

5. Database & Data Layer (Weeks 5-12)

5.1 Database Design

- 5.1.1 Schema creation (events, issues, users, comments, votes)
- 5.1.2 Relationships and constraints
- 5.1.3 Indexes and performance tuning
- 5.1.4 Data migration scripts

5.2 Data Management

- 5.2.1 Seed data for development and staging
- 5.2.2 Backup and recovery procedures
- 5.2.3 Data retention policies

5.3 Geographic Data

- 5.3.1 Geolocation data setup
- 5.3.2 Map tile server configuration

6. Integration & Third-Party Services (Weeks 8-12)

6.1 External APIs

- 6.1.1 Calendar/scheduling API integration
- 6.1.2 Geolocation/maps API (Google Maps, Mapbox)
- 6.1.3 Email service integration
- 6.1.4 Social media sharing integration

6.2 Payment Processing (if needed)

- 6.2.1 Payment gateway setup
- 6.2.2 Transaction handling

7. Testing & QA (Weeks 10-14)

7.1 Unit Testing

- 7.1.1 Frontend component tests
- 7.1.2 Backend API tests
- 7.1.3 Utility function tests
- 7.1.4 Target: >80% code coverage

7.2 Integration Testing

- 7.2.1 API and database integration tests
- 7.2.2 Frontend-backend integration tests
- 7.2.3 Third-party service integration tests

7.3 End-to-End Testing

- 7.3.1 User workflow testing
- 7.3.2 Critical path scenarios
- 7.3.3 Cross-browser E2E tests

7.4 Performance Testing

- 7.4.1 Load testing
- 7.4.2 Stress testing
- 7.4.3 Database query optimization

7.5 Security Testing

- 7.5.1 OWASP vulnerability scan
- 7.5.2 Input validation testing
- 7.5.3 XSS and CSRF testing
- 7.5.4 Rate limiting validation

7.6 Manual QA

- 7.6.1 Functional testing on all features
- 7.6.2 Usability testing
- 7.6.3 Accessibility testing (WCAG 2.1 AA)
- 7.6.4 Browser compatibility matrix

7.7 Bug Tracking & Fixes

- 7.7.1 Issue logging and prioritization
- 7.7.2 Regression testing
- 7.7.3 Release candidate validation

8. DevOps & Infrastructure (Weeks 8-14)

8.1 Infrastructure Setup

- 8.1.1 Hosting provider selection
- 8.1.2 Server provisioning and configuration
- 8.1.3 Database hosting setup

8.1.4 CDN configuration

8.2 CI/CD Pipeline

8.2.1 Automated build process

8.2.2 Automated testing in pipeline

8.2.3 Staging environment deployment

8.2.4 Production deployment automation

8.3 Monitoring & Observability

8.3.1 Application performance monitoring (APM)

8.3.2 Error tracking (Sentry, etc.)

8.3.3 Log aggregation

8.3.4 Uptime monitoring

8.3.5 Alert configuration

8.4 Security Hardening

8.4.1 SSL/TLS certificates

8.4.2 Firewall rules

8.4.3 DDoS protection

8.4.4 Regular security patches

9. Documentation & Knowledge Transfer (Weeks 12-15)

9.1 User Documentation

9.1.1 User guide and tutorials

9.1.2 FAQ section

9.1.3 Video walkthroughs

9.1.4 Help center content

9.2 Technical Documentation

9.2.1 API documentation (Swagger/OpenAPI)

9.2.2 Architecture and deployment guide

9.2.3 Database schema documentation

9.2.4 Code repository README

9.2.5 Troubleshooting guides

9.3 Admin Documentation

9.3.1 Admin panel user guide

9.3.2 Moderation guidelines

9.3.3 System administration procedures

10. Deployment & Launch (Weeks 14-16)

10.1 Pre-Launch

10.1.1 Staging environment validation

10.1.2 Load testing in production-like environment

10.1.3 Security audit finalization

10.1.4 Backup and disaster recovery testing

10.1.5 Launch checklist review

10.2 Soft Launch / Beta

10.2.1 Limited user testing (beta group)

10.2.2 Monitoring and feedback collection

10.2.3 Issue triage and quick fixes

10.3 Full Launch

10.3.1 Production deployment

10.3.2 Marketing and announcements

10.3.3 Social media campaigns

10.3.4 Press releases

10.4 Launch Day Operations

10.4.1 Incident response readiness

10.4.2 Real-time monitoring

10.4.3 Support team on standby

10.4.4 Issue tracking and communication

11. Post-Launch Support & Maintenance (Ongoing)

11.1 Bug Fixes & Hotfixes

11.1.1 Critical bug response

11.1.2 Regular patch releases

11.2 Performance Monitoring

11.2.1 Daily uptime checks

11.2.2 Performance metrics review

11.2.3 Optimization iterations

11.3 User Support

- 11.3.1 Ticketing system management
- 11.3.2 FAQ updates based on feedback
- 11.3.3 User feedback collection

11.4 Feature Enhancements (Phase 2+)

- 11.4.1 Prioritize feature requests
- 11.4.2 Plan next iteration
- 11.4.3 Roadmap updates

12. Training & Enablement (Weeks 14-16)

12.1 Internal Training

- 12.1.1 Support team training
- 12.1.2 Admin/moderation training
- 12.1.3 Operations team training

12.2 External Training

- 12.2.1 Community partner training
- 12.2.2 Public documentation launch

Key Milestones

Milestone	Target Date	Deliverables
Design Complete	End of Week 4	Design system, wireframes, specs
Alpha Release	End of Week 10	Core features working, internal testing
Beta Release	End of Week 13	Feature complete, external testing
Production Launch	End of Week 16	Live deployment, monitoring active

Resource Allocation by Phase

Phase	Duration	Team Composition
Planning & Setup	Weeks 1-2	Full team (16 people)
Development	Weeks 5-12	Frontend (4), Backend (4), QA (2), DevOps (1), Other (5)
Testing & QA	Weeks 12-14	QA (4), Dev Support (2), Other (10)
Launch & Operations	Weeks 14-16	DevOps (2), QA (2), Support (1), Management (11)